

Project Decision Log

This document tracks all the major decisions, queries, and architectural changes we make throughout the project, explaining **what** we chose and **why**.

1. Migration to Real Codeforces API

User Query: Replace synthetic data with real-time Codeforces (CF) API data for recommendations.

Decision: We decided to drop the old synthetic data generation scripts entirely. Instead, we created `fetch_cf_data.py` to pull the real 10,000+ problemset from Codeforces, as well as real users and their submissions. **Why:** Recommending real problems based on real submission history creates a vastly more useful and realistic MLOps project.

2. Rebuilding the Project "From Scratch"

User Query: Begin the model from scratch and explain every file so it's clear where to start. **Decision:** Created a `project_architecture.md` guide that maps out the entire new system. We are systematically rewriting each component (Database → Data Fetcher → ML Model → API Backend → Deployment). **Why:** To ensure complete understanding of the architecture, since integrating live CF data fundamentally changes how the original project worked.

3. Generalizing the ML Model Features

Decision: When rewriting `scripts/train.py`, we dropped the old method of using "label-encoded User IDs and Problem IDs". Instead, we are training the model using general features: User Rating, Problem Rating, and Problem Tags. **Why:** If we used hardcoded User IDs, the model would only work for the 50 users we downloaded. By using ratings and tags, the model learns general rules (e.g., "1500 rated users struggle with 1700 rated math problems") and can make predictions for *any* live Codeforces user in real-time.

4. Periodic Data Collection Strategy

User Query: How can we load this data periodically rather than just at the beginning? **Options Considered:**

- AWS EventBridge (Cloud Cron)
- APScheduler inside the FastAPI app
- Docker Compose Cron Service **Decision:** We chose the **Docker Compose Cron Service**. We will add a lightweight scheduler container to our `docker-compose.yml` that periodically runs the data fetching script. **Why:** It is the safest, simplest option that keeps the project fully portable. You can run `docker compose up` on your local laptop, and it will behave exactly the same as it does when deployed on AWS.

5. Deployment Troubleshooting

During the final Docker deployment, we encountered and fixed three notable bugs:

Bug 1: Missing `requests` Dependency

- **Problem:** The `recommender_app` container crashed with `ModuleNotFoundError: No module named 'requests'`.
- **Why it occurred:** We added new code to hit the live Codeforces API (using the `requests` library) but forgot to update the Docker `requirements.txt`.

- **Solution:** Added `requests` to `requirements.txt` and rebuilt the containers using the `--build` flag.

Bug 2: Database Foreign Key Violation (`gkpj`)

- **Problem:** The background scheduler crashed with `psycopg2.errors.ForeignKeyViolation` when inserting submissions.
- **Why it occurred:** Codeforces has many unrated or "Gym" problems. We intentionally skipped saving these problems to our database because the ML model needs problem ratings to train. However, when downloading user submissions, we tried to save submissions for those missing problems, causing a foreign key mismatch.
- **Solution:** Modified `scripts/fetch_cf_data.py` to cross-reference the problem ID with the local database *before* saving a submission.

Bug 3: Pickling Error (`AttributeError: Can't get attribute 'split_tags'`)

- **Problem:** The FastAPI web server threw a 500 Internal Server Error when trying to load the Machine Learning model (`logistic_model.joblib`).
- **Why it occurred:** The custom tokenizer function (`split_tags`) was defined directly inside the `train.py` script. When `joblib` saved the model, it tied the function to the `__main__` namespace. When the web server (`uvicorn`) tried to load the model, its `__main__` was `uvicorn`, causing a crash.
- **Solution:** Moved `split_tags` out of the training script and into `app/model.py` . This allowed both the training script and the web server to import the function from a shared, consistent module namespace. Retraining the model fixed the issue.

Bug 4: Scikit-Learn Version Mismatch (`AttributeError: 'LogisticRegression' object has no attribute 'multi_class'`)

- **Problem:** After fixing Bug 3, the web server threw another 500 Internal Server Error with a `multi_class` missing attribute.
- **Why it occurred:** The ML model was inadvertently retrained directly on the host machine using `scikit-learn` version 1.5+, rather than inside the Docker container. When the older `scikit-learn==1.4.1` running inside the Docker container attempted to load the new `.joblib` file, the version mismatch caused the framework to crash trying to find attributes that were renamed/removed.
- **Solution:** Re-ran the training script directly inside the Docker container (`docker exec recommender_app python scripts/train.py`) to ensure the model was trained using the exact Python environment and dependency versions it would be loaded in. The web server immediately picked up the correctly formatted model.

6. Frontend UI and Hot-Reloading

- **Decision:** Built a modern, glassmorphism UI directly served by FastAPI using `StaticFiles` , and mounted `./frontend:/app/frontend` in `docker-compose.yml` .
- **Why:** To make the API accessible via a browser. Using a simple HTML/CSS/JS stack prevents the complexity of Node.js builds. Mounting the volume enables hot-reloading, so CSS and HTML changes instantly appear upon browser refresh without needing to rebuild the Docker container.

7. Real-Time Solved Problem Filtering

- **User Query:** If a user solves a recommended problem, do they have to wait 24 hours for the background task to remove it?

- **Decision:** No, the problem is removed instantly. The background cron task is only used to scrape massive amounts of global data to *retrain* the AI model. The actual recommendation inference engine (`app/recommender.py`) fetches the user's personal submission history *live* from the Codeforces API every single time the "Find Problems" button is clicked, ensuring the `solved_ids` list is perfectly up to date.

8. Stratified Recommendations & The "Identical Problems" Bug

- **User Query 1:** Categorize the top 10 recommendations into Easy (>75%), Medium (50-75%), and Hard (<50%), but it was originally only returning Medium problems.
- **Why it only returned Medium:** The backend was originally limiting the candidate search space to `500` problems to save RAM. The SQL query simply fetched the first 500 problems chronologically. Since those older problems happened to have homogenous difficulties, the ML model predicted ~50-60% win-rate for all of them.
- **Initial Fix:** We increased the candidate search limit from `500` to `2000` to force the ML model to evaluate a much broader variance of problems, successfully allowing it to find highly-probable (Easy) and low-probable (Hard) problems.
- **User Query 2:** Why are different users (e.g. `kani2315` and `JaiJerry23`) getting the exact same problem recommendations?
- **Why they were identical:** Because we were using `limit(2000)` without any randomized or specific sorting, the database returned the exact same block of 2000 problems for every user. If two users had similar profiles (e.g., both were Unrated), the ML model scored that identical block of 2000 problems the exact same way, resulting in identical top 30 lists.
- **Final Solution:** We completely removed the `limit()` from the database query in `app/recommender.py` . Now, the backend deterministically feeds *all* 8000+ unsolved Codeforces problems through the ML model in a fraction of a second. This guarantees that every user gets evaluated against the entire database, ensuring they see their absolute best mathematical matches. It also ensures that the recommendations are completely consistent for the same user if they refresh the page.

9. Frontend Recommendation Filters

- **Decision:** Added client-side JavaScript filters (All, Warm Up (Easy), Medium, Challenge (Hard)) to the UI.
- **Why:** Since the API now returns 30 problems (10 of each category), the filters allow the user to easily toggle between problem types based on what kind of practice they want to do that day, without needing to make additional backend API calls.

10. Dynamic Target Proximity Selection

- **User Query:** A user with 956 rating only got 1 Easy problem, and a Pupil and Specialist received the exact same problem sets.
- **Why it occurred:** Because the model evaluated the entire DB and sorted by *absolute highest probability*, it simply gave everyone the absolute easiest problems in the database (800-rated problems). Additionally, because of strict `> 0.70` thresholds, a lower-rated user might not have mathematically had 10 problems that crossed that strict absolute threshold.
- **Decision:** Replaced strict percentage bounds with a "Dynamic Target Proximity" algorithm.
- **How it works:** The engine now sorts all problems by how close they are to a specific *target* probability.
 - **Easy:** Finds the 10 problems mathematically closest to `P = 0.75` for that specific user.
 - **Medium:** Finds the next 10 problems closest to `P = 0.50` .

- **Hard:** Finds the next 10 problems closest to `P = 0.25`.
- **Result:** This perfectly scales to any user's rating curve. A Specialist's "Easy" problem will be much harder than a Pupil's "Easy" problem, ensuring completely personalized recommendations. It also guarantees every user will receive exactly 10 problems in each category, regardless of their rating.

11. Railway Cloud Deployment Architecture

- **Decision (Platform):** Migrated deployment from AWS EC2 to Railway.
 - **Why:** Railway natively connects to GitHub, automatically manages Docker builds, provisions SSL certificates, and offers managed PostgreSQL databases out-of-the-box. This removes the need for complex `deploy_aws.sh` User-Data scripts and manual server maintenance.
- **Decision (Volume):** Attached a persistent Railway Volume to the `/app/models` directory.
 - **Why:** Railway containers are ephemeral; if the web server restarts, any files generated during runtime are wiped. By attaching a permanent Volume, the Machine Learning model (`logistic_model.joblib`) that the background task spends minutes training is safely preserved across server restarts and deployments.
- **Decision (Background Data Fetcher):** Kept the background data fetcher and model trainer running in an infinite 24-hour loop.
 - **Why:** The Codeforces problemset and user submission data changes every day. The background task is strictly required to download the massive global 8,000+ problem dataset, populate the Postgres database, and continuously retrain the ML model on fresh data so the recommendations remain accurate.
- **Decision (Embedded Scheduler):** Embedded the background ML training loop directly inside the FastAPI web server using Python `asyncio` background tasks, rather than running it as a completely separate service.
 - **Why:** Railway does not natively allow two different services to mount and share the exact same Volume simultaneously. If the scheduler ran as a separate service, it would save the trained model to its own isolated volume, and the FastAPI web server would never be able to access the `.joblib` file. By embedding the scheduler directly inside the FastAPI app, they share the exact same runtime environment and the exact same Volume, seamlessly solving the file-sharing constraint while cutting the hosting costs in half!

12. Recommendation Engine Upgrade (XGBoost & Smart Features)

- **Decision (Model):** Swapped `LogisticRegression` for an `XGBClassifier` (XGBoost).
 - **Why it's better:** Logistic Regression struggles with non-linear relationships (e.g., a user might be great at high-rating Math problems, but terrible at low-rating Graph problems). XGBoost is an advanced decision-tree algorithm that excels at finding these hidden, non-linear patterns. Our local tests proved this by bumping accuracy from 67% to over 70.5%.
- **Decision (Smart Features):** Added `problem_solve_count` and `user_total_solved` as core numerical features to the ML training pipeline.
 - **Why it's better:** Previously, the model didn't know if a problem was obscure or incredibly popular. By parsing the `problemStatistics` array from the Codeforces API, we now feed the global popularity of every problem into the ML engine, inherently biasing it toward higher-quality, widely-practiced problems. Tracking user activity (`user_total_solved`) also helps the model differentiate between veterans and beginners.
- **Decision (Unrated User Handling):** Implemented an "Estimated Rating" algorithm in `app/recommender.py`.

- **How it works:** If an unrated user (rating = 0) requests recommendations, the system scans all the problems they *have* solved, fetches the official Codeforces difficulty rating of those problems, and computes the mathematical average.
- **Why it's better:** Previously, passing 0 into the ML model would ruin the predictions because it expected a standard rating (~1500). By calculating their true hidden skill level on the fly, the XGBoost model can instantly provide perfect recommendations even if the user has never competed in a rated contest.

13. The Core Recommendation Algorithm (How it actually works)

To ensure the recommendations are genuinely useful (and not just blindly suggesting popular problems), the system uses a multi-step pipeline combining statistical analysis with Machine Learning:

1. **Submission Analysis:** It downloads all of the user's past Codeforces submissions.
2. **Global Win-Rate:** It calculates the user's **Overall Win Rate** across all problems (e.g., 50%).
3. **Topic Grouping:** It groups every single submission by its topic/tag (e.g., Math, Graphs, Greedy).
4. **Weak Tag Identification:** If a user's win rate in a specific topic (e.g., Graphs is 20%) is significantly lower than their overall win rate (50%), that topic is flagged as a **Weak Tag**.
5. **Candidate Filtering:** It searches the database for thousands of completely unsolved problems that specifically contain those Weak Tags.
6. **XGBoost Inference:** Finally, it feeds those candidates into the XGBoost Machine Learning model. The model computes a mathematical "Solve Probability" for each problem based on the user's rating, problem rating, problem popularity, and user activity.
7. **Dynamic Proximity:** It doesn't just return the easiest problems. It mathematically selects the exact problems closest to $P=0.75$ (Easy), $P=0.50$ (Medium), and $P=0.25$ (Hard) so the user gets a perfect mix of practice difficulties targeting their worst topics!

14. The End-to-End MLOps Pipeline

To understand the complete "line" or lifecycle of how data flows through this system:

- **Data Ingestion (`fetch_cf_data.py`):** Connects to the Codeforces API, downloads the global problemset (including `solvedCount`), and fetches 30,000+ real user submissions. Stores this raw data cleanly in PostgreSQL.
- **Model Training (`train.py`):** Queries the PostgreSQL database to join Users, Problems, and Submissions. It engineers new features (like user total solved) and trains the `XGBClassifier` . The final, trained pipeline is exported as a binary `.joblib` file.
- **Continuous Integration (Embedded Scheduler):** Runs inside `app/main.py` . Every 24 hours, it triggers the Data Ingestion script followed immediately by the Model Training script in a background thread, ensuring the model never goes stale.
- **Real-Time Inference (`recommender.py`):** The FastAPI server loads the `.joblib` file into memory. When a user requests a recommendation, it analyzes their live Codeforces profile, generates the candidate features, and runs them through the in-memory XGBoost model to serve sub-second predictions.

15. Branching & Deployment Strategy

- **Decision:** Major architectural upgrades (like swapping the entire ML engine from Logistic Regression to XGBoost) are committed to a separate Git branch (e.g., `feature/xgboost-upgrade`) rather than pushing directly to `main` .
- **Why:** Pushing directly to `main` immediately triggers a live production deployment on Railway. By using a separate branch, we can test the new ML model locally, ensure the UI doesn't break, and

merge it into production only when we have 100% confidence.

16. Profile Analysis Dashboard

- **User Query:** Provide a profile analysis section showing current rating, rating practice range (± 300), rating-wise solved stats, and topic-wise strong/weak stats in a separate view.
- **Decision:** Built a dynamic Profile Analysis Dashboard seamlessly integrated into the existing Single Page Application (SPA). Added a new `/analysis/{handle}` backend endpoint to compute these statistics on the fly.
- **Why it's better than a new HTML page:** Navigating to a separate `.html` page causes a full browser reload, which disrupts the premium user experience. By implementing a JavaScript toggle, the problem cards fade out and the analysis dashboard fades in instantly, making the website feel like a highly polished, modern web app.
- **Features Implemented:**
 1. **Rating Constraint Filter:** Implemented a hard constraint (`user_rating - 300` to `user_rating + 300`) directly in the SQL candidate query. This fixes the bug where XGBoost occasionally suggested mathematically impossible problems (like a 2700 rated problem for a 1200 user) simply because they were the "closest" available options in the dataset.
 2. **Topic Analytics:** Calculated the precise win-rate (`solved / attempts`) for every topic. The UI highlights the Top 5 Strongest topics in Emerald Green and the Top 5 Weakest topics in Crimson Red.
 3. **Rating Distribution:** Plotted a CSS-based bar chart showing the historical distribution of problems the user has solved across all Codeforces difficulty tiers.

17. Real-Time Adaptive Difficulty (The Feedback Loop)

- **User Query:** If a user solves 10 DP questions in a single day, do they have to wait 24 hours for the system to suggest harder DP problems?
- **Decision:** No, the system adapts instantly. The 24-hour background scheduler is *exclusively* used to fetch global database updates (newly published problems and the submissions of the other 30,000 users) to retrain the underlying XGBoost model's general intelligence.
- **How it works:** When a user clicks "Find Problems", their *personal* profile stats are fetched live from the Codeforces API. If they just solved 10 DP problems, their total solve count instantly increases, and their DP win-rate instantly goes up. The XGBoost model dynamically evaluates these new live stats and immediately predicts a higher win-probability for DP problems. To keep the user in the optimal 50% "Challenge" sweet spot, the system is mathematically forced to serve them much harder DP problems immediately. This creates a completely real-time, personalized feedback loop.

18. Analytics and Recommendation Refinements

Over the final review cycles, we made several critical algorithmic tweaks to ensure the Profile Analysis Dashboard and the ML Recommendation Engine matched perfectly:

1. The "Limbo State" Bug Fix

- **Problem:** If a user attempted a topic 1 to 4 times and solved 0 problems, the topic vanished from the dashboard entirely. It wasn't considered "Weak" (which required ≥ 5 attempts to prove statistical weakness) and it wasn't considered "Avoided" (which strictly required 0 attempts).
- **Solution:** Redefined the logic so that any topic with `< 5` attempts and `0` solves is officially classified as an "Unexplored/Avoided Topic" and pushed to that list.

2. Rating Distribution Chart Accuracy

- **Problem:** The CSS bar chart showing the user's solved problem distribution (e.g., 50 problems at 800 rating, 20 at 900) was missing problems and didn't match the official Codeforces profile graph.
- **Why:** We were counting the ratings by looking up the user's solved IDs in our local Postgres database, which only contains a subset of all Codeforces problems.
- **Solution:** Updated the backend to extract the problem rating directly from the live Codeforces API submission payload. This guarantees the chart is 100% perfectly synchronized with Codeforces data. (Additionally, fixed a CSS flexbox `align-items: flex-end` bug that collapsed the bars).

3. Proportional "Avoidance Ratio" Algorithm

- **Problem:** For highly experienced users (like Grandmasters) who have attempted every single popular topic at least once, the "Avoided Topics" list was forced to display ultra-rare, obscure topics (like `2-sat` or `fft`). Because there are barely any problems for those tags, the ML engine couldn't find suggestions for them and triggered a fallback that injected random problems (like `math` or `greedy`).
- **Solution:** Eliminated the strict `0 attempts` requirement. The algorithm now calculates a mathematical **Avoidance Ratio** for every topic: $\text{Global Frequency} / (\text{User Attempts} + 1)$.
- **Why it's brilliant:** Massive topics (like `dp` with 10,000 problems) where a user only has 10 attempts will now yield a massive score and shoot to the top of the Avoided list. It dynamically ranks tags based on how much the user avoids them relative to their global popularity, eliminating obscure tags from ruining the suggestions.

4. Per-Tag Candidate Selection (Eliminating "Drowning Out")

- **Problem:** A user noticed that `hashing` (their #1 weakest topic) was completely missing from the "Let's Improve These" ML recommendations.
- **Why:** The backend grouped all 5 Weak topics into a single massive SQL `OR` query. Because a weak topic like `constructive algorithms` has 500+ problems in the database and `hashing` only has 27, the massive topics mathematically drowned out the rare ones when the ML model picked the top 15 results.
- **Solution:** Completely rewrote the recommendation loop. It now iterates through the Weak and Avoided tags **one-by-one**. It fetches candidates, scores them with XGBoost, and picks the top 3 best problems for *every single tag individually*. This guarantees perfectly even distribution across all weak/avoided topics, ensuring no tag is ever drowned out again.

19. Embedded Background Scheduler (vs. Standalone CRON)

- **User Query:** "Why do I no longer need to set up a separate background service for ML training in Railway, when earlier it was a separate script?"
- **The Old Way:** In early iterations (and traditional AWS/EC2 setups), ML pipelines were completely decoupled from the Web API. You would have to deploy two separate servers: one running the FastAPI web server, and a second separate "Worker" server running a CRON job to trigger `train.py` every 24 hours.
- **The Problem:** Modern PaaS cloud hosts (like Railway or Heroku) charge you *per running service*. Spinning up a second container just to run a Python script once a day doubles your monthly hosting costs and drastically increases infrastructure complexity (requiring Redis, Celery, or CRON configurations).
- **The Solution:** We radically simplified the architecture by embedding the ML pipeline directly into FastAPI's native `asyncio` event loop.
- **How it works:** When FastAPI boots up, it executes `asyncio.create_task(background_scheduler())`. This spawns an immortal background

thread *inside* the exact same web container. This thread silently triggers `fetch_cf_data.py` and `train.py` using `subprocess.exec` without ever blocking the web API from serving user requests.

- **The Result:** Zero extra setup, zero extra cloud hosting costs, and the system is entirely self-contained within a single Docker container.

20. Weighted Strength Score Heuristic

- **Problem:** Originally, the "Strongest Topics" dashboard panel sorted topics strictly by win-rate (`solved / attempts`). A user noticed a major flaw: a topic with 7 solves and 9 attempts (77% win rate) mathematically outranked a massive topic with 100 solves and 200 attempts (50% win rate). A sample size of 7 is too small to declare it a "Strong" topic over one where the user has invested massive time and effort.
- **Rejected Alternatives:**
 1. *Set a strict minimum threshold (e.g., must have ≥ 20 solves).* Rejected because it rigidly punishes newer users who might legitimately have a strong topic with 15 solves, forcing their dashboard to look empty.
 2. *Sort strictly by total solves.* Rejected because it ignores efficiency completely. If a user solved 50 DP problems out of 500 attempts (10% win rate), it would still show up as a "Strong" topic just because of high volume, which is factually incorrect.
- **Chosen Solution:** Implemented a continuous mathematical heuristic: `strength_score = solved * win_rate` .
- **Why it's brilliant:** It perfectly balances volume and efficiency. The 100-solve topic (50% win rate) yields a score of `50.0` . The 7-solve topic (77% win rate) yields a score of `5.39` . The 100-solve topic beautifully outranks the low-volume topic.
- **Weak Topic Tie-Breaker:** Conversely, "Weak Topics" are strictly sorted by the lowest win-rate. To break ties (e.g., two topics with a 10% win rate), the algorithm now ranks the one with the *highest* attempts as the weaker topic, because the weakness is more statistically proven.

21. Fixing SQLAlchemy Dict Type Error

- **Problem:** When fetching recommendations, the backend unexpectedly crashed with (psycopg2.ProgrammingError) can't adapt type 'dict' .
- **Why it occurred:** In the previous update, we changed the `_get_avaoided_tags` function to return dictionaries (containing both the tag string and the attempt count) so the frontend could dynamically render "X attempts" instead of a hardcoded "0 attempts". However, the internal `recommend()` function was still trying to pass these dictionary objects directly into a SQLAlchemy `.filter(tags.contains())` query, causing PostgreSQL to crash because it expected a pure string.
- **Solution:** In `recommend()` , we now explicitly extract the string `"tag"` from the dictionaries returned by `_get_avaoided_tags` before building the database queries. This perfectly isolates the logic: the frontend still gets the rich dictionary data for the dashboard, while the database queries securely receive valid string data.

22. Technology Stack

Date: 2026-08-21 **Context & Decision:** The following technology stack was deliberately chosen to balance performance, machine learning integration, and ease of cloud deployment:

- **FastAPI (Python):** Chosen as the core web framework due to its native asynchronous capabilities (`asyncio`), high performance, and seamless integration with Python's premier data science

libraries.

- **XGBoost:** Selected as the Machine Learning engine for scoring/ranking problem recommendations over Logistic Regression, due to its vastly superior performance on tabular data and non-linear relationships.
- **PostgreSQL:** Production relational database hosted on Railway, selected for robust concurrent read/writes during ML training. (SQLite is used as a local fallback).
- **SQLAlchemy (ORM):** Used for database abstraction, schema management, and seamless querying.
- **Pandas & NumPy:** Utilized heavily in `scripts/train.py` for high-speed data manipulation and feature engineering before feeding into the ML model.
- **scikit-learn & joblib:** Used for ML pipeline preprocessing and efficient serialization (`.joblib` format) of the trained XGBoost model.
- **Vanilla HTML / CSS / JS:** Chosen for the frontend to eliminate heavy build steps (like Webpack/Node.js). It is served instantly via FastAPI's `StaticFiles` , yet heavily stylized with modern CSS glassmorphism and dynamic DOM manipulation.
- **Uvicorn:** The ASGI server used to run the FastAPI application.
- **Docker:** Used to containerize the entire MLOps pipeline and web server into a single, reproducible environment.
- **Railway (PaaS):** Chosen for cloud deployment. It automatically handles the Docker builds, PostgreSQL provisioning, and provides instant SSL domain routing.
- **Prometheus:** Integrated for backend metrics and observability.

23. Stratified Sampling for ML Data Collection

Date: 2026-08-20

Context:

The background script (`scripts/fetch_cf_data.py`) pulls 50 active Codeforces users from the official API to build the training dataset for the XGBoost model. By default, the `user.ratedList` API returns users sorted by their rating (highest to lowest). Our initial code simply sliced the first 50 users (`users_data[:50]`), meaning the machine learning model was exclusively trained on the solving patterns of Legendary Grandmasters (ratings 2800+). This created a severe selection bias, causing inaccurate `P(solve)` predictions for beginners and intermediate users using the Recommender API.

Decision:

We implemented **Stratified Sampling**. Instead of grabbing the top 50, the script now categorizes the active users into 5 distinct skill buckets:

- Beginner (<1200)
- Pupil (1200 - 1399)
- Specialist (1400 - 1599)
- Expert (1600 - 1899)
- Advanced (>=1900)

It then uses `random.sample()` to pull exactly 10 users from each bucket, resulting in 50 evenly distributed users. We also added a 15-second `timeout` to the requests to prevent the script from hanging indefinitely during Codeforces API stalls.

Impact & Results:

- **Before:** The model was heavily skewed and only understood the problem-solving behavior of top-tier coders.
- **After:** The script successfully pulled 23,127 diverse submissions from a balanced mix of beginners to grandmasters. The XGBoost model trained on this dataset achieved an **Accuracy, Precision, Recall, and F1 Score of ~0.66** (66%). This provides a much more calibrated and realistic prediction

matrix, allowing the application to successfully recommend "Zone of Proximal Development" (ZPD) problems to coders of all skill levels.

24. Complete End-to-End System Flow (How Everything Happens)

Date: 2026-08-21 **Context:** To ensure absolute clarity on how the entirely automated architecture operates from start to finish.

Here is the exact sequential flow of the entire application once deployed on Railway:

Phase 1: Server Initialization (The Boot Up)

1. Railway detects a GitHub push and spins up a fresh Docker container.
2. The Docker container runs `uvicorn app.main:app` which starts the FastAPI web server.
3. Upon startup, FastAPI immediately fires an asynchronous background task (`asyncio.create_task`) that operates invisibly alongside the web server.

Phase 2: Background Data Collection & ML Training (The "Brain" Building)

1. The background task triggers `scripts/fetch_cf_data.py`.
2. This script reaches out to the live Codeforces API and downloads the latest global 10,000+ problemset.
3. It uses **Stratified Sampling** to select 50 diverse users (Beginners to Grandmasters) and downloads their latest 20,000+ problem submissions.
4. It saves all of this raw data into the live **PostgreSQL** database.
5. Immediately after fetching, it triggers `scripts/train.py`.
6. `train.py` queries PostgreSQL, joins the tables, and engineers features (like `user_total_solved`).
7. It feeds this massive dataset into the **XGBoost Algorithm** which learns the hidden mathematical patterns of what makes a coder succeed or fail at a specific problem.
8. The trained model is serialized into a `.joblib` file and saved to a **Persistent Railway Volume** so the "brain" is never lost if the server restarts. (This entire Phase 2 loop repeats every 24 hours).

Phase 3: The User Experience (Real-Time Inference)

1. A user visits the frontend webpage and enters their Codeforces handle.
2. The frontend shoots an API request to the FastAPI backend (`/recommend/{handle}`).
3. The backend instantly reaches out to the Codeforces API to download the user's *live* personal profile and submission stats.
4. It analyzes their submissions to mathematically calculate their **Strong Topics**, **Weak Topics**, and **Avoided Topics** based on global popularity vs. personal avoidance ratios.
5. The backend queries PostgreSQL for thousands of unsolved problems targeting their exact weaknesses.
6. It passes these thousands of candidates into the in-memory **XGBoost Model** (loaded from the `.joblib` file).
7. The model evaluates every candidate and outputs a "Predicted Solve Probability".
8. The **Dynamic Proximity Algorithm** selects the 10 problems mathematically closest to 75% (Easy), 50% (Medium), and 25% (Hard) relative to the user's personal rating.
9. FastAPI returns these perfectly curated problem sets (and the Profile Dashboard statistics) to the frontend as JSON.
10. The Vanilla JS frontend dynamically renders the glowing Glassmorphism UI cards, allowing the user to seamlessly toggle between difficulties!

25. Deep Dive: The Complete Machine Learning Pipeline

Date: 2026-08-21 **Context:** Explaining exactly how the `train.py` script transforms raw SQL tables into a highly intelligent recommendation engine.

The Machine Learning architecture follows a strict, highly engineered pipeline designed to maximize predictive accuracy for unrated, beginner, and grandmaster users alike:

Step 1: Feature Engineering (The Inputs)

The model does *not* memorize specific user names or problem names. Instead, it extracts five generalized numerical and categorical features for every single submission:

1. `user_rating` : The Codeforces rating of the user. (Unrated users are dynamically estimated).
2. `user_total_solved` : The sheer volume of problems a user has completed (helps the model differentiate between veterans and newcomers).
3. `problem_rating` : The official difficulty rating of the problem.
4. `problem_solve_count` : The global popularity of the problem (biases the model toward highly-practiced, high-quality problems).
5. `tags` : A comma-separated list of topics (e.g., `"math, greedy, dp"`).

Step 2: Categorical Preprocessing (The MultiLabelBinarizer)

Machine learning models cannot read raw strings like `"math, dp"` . To solve this, the pipeline uses a custom `split_tags` function integrated with scikit-learn's `MultiLabelBinarizer` .

- This automatically scans all 35+ possible Codeforces topics and creates a binary column for each one (e.g., `tag_math = 1` , `tag_dp = 1` , `tag_graphs = 0`).
- This allows the XGBoost model to mathematically isolate and learn exactly which topics are inherently harder than others.

Step 3: Dataset Assembly & Splitting

- **The Target (Y):** The label the model is trying to predict is `solved` (1 if the user solved it, 0 if they failed or gave up).
- **Train/Test Split:** The massive dataset of 23,000+ submissions is shuffled and split (80% for Training, 20% for Testing) to ensure the model doesn't overfit and can successfully generalize to problems it has never seen before.

Step 4: The XGBoost Algorithm (The Brain)

We utilize the `XGBClassifier` (Extreme Gradient Boosting), which builds hundreds of sequential decision trees where each tree corrects the mathematical errors of the previous one.

- **Why XGBoost over Logistic Regression?** Logistic Regression assumes relationships are perfectly straight lines. But human skill isn't linear! A user might have a 90% win rate on 1500-rated Math problems, but a 10% win rate on 1500-rated Graph problems. XGBoost excels at capturing these highly complex, non-linear, multi-variable interactions.
- During training, XGBoost mathematically discovers rules like: *"If User Rating > Problem Rating by 200 points, AND tag = 'math', AND User Total Solved > 500, then Solve Probability = 96%."*

Step 5: Serialization & Inference (`predict_proba`)

1. The fully trained `MultiLabelBinarizer` and the `XGBoost Classifier` are bundled together into an unbreakable `Pipeline` object.

2. This Pipeline is serialized and saved to disk as `logistic_model.joblib`.
3. During Real-Time Inference, the backend loads this file. Instead of asking the model for a strict Yes/No prediction, it calls `predict_proba()`.
4. `predict_proba()` returns the exact decimal probability (e.g., `0.7512`). This continuous mathematical decimal is what allows the Dynamic Proximity Algorithm to perfectly sort and categorize problems into Easy, Medium, and Hard targets!

26. Complete MLOps Cycle & Kubernetes Alternatives

Date: 2026-09-03 **Context:** Documenting the alternative tools used instead of a complex Kubernetes setup for orchestrating the MLOps pipeline.

To avoid the operational overhead of maintaining a full Kubernetes cluster (like EKS or GKE) and complex orchestration tools (like Kubeflow or Airflow), we architected a lightweight, containerized alternative:

1. Local Orchestration: Docker Compose

Instead of writing Kubernetes manifests (Deployments, Services), we use `docker-compose.yml` to define our multi-container environment locally. This includes:

- The PostgreSQL database.
- The FastAPI application (backend/frontend).
- A custom `scheduler` container for background ML tasks.

2. Production Deployment: Railway PaaS

We deploy the application using **Railway** (Platform-as-a-Service) instead of a Kubernetes cluster. Railway natively handles:

- Automated container builds from GitHub pushes.
- Persistent volumes (crucial for storing the `.joblib` model and PostgreSQL data).
- Service routing and load balancing without the need for K8s Ingress controllers.

3. Workflow Orchestration: Background Tasks vs CronJobs

Instead of using Kubernetes CronJobs for daily ML training, we utilize:

- **Locally:** A `scheduler` container running an infinite Bash loop (`while true; do python scripts/fetch_cf_data.py && python scripts/train.py; sleep 86400; done`).
- **Production (Railway):** FastAPI's `asyncio.create_task()` to trigger a background loop that invisibly handles data fetching and model training asynchronously alongside the web server.

This approach delivers the core benefits of containerization and CI/CD without the massive complexity of Kubernetes.